

Exercise Sheet 4

November 7th: JUnit

Note: You can use JUnit (V $\leq$ 3.8) for exercise 2 and 3 or (better) my source code of the core classes for *all* exercises of this sheet!

Exercise 1

Fill the gaps in my source code of the core classes of Junit Code (as programmed on the fly in the class room lecture, see `junitonthe-fly.zip` in Moodle).

Exercise 2

a) Extend JUnit with Conditional Test Cases:

1. Implement a `ConditionalTestCase` extending `TestCase` to support a `public boolean shouldRun()` method. Subclasses can override the default and henceforth developers can ex- or include certain test cases depending on context (i.e. for different platforms etc.)
2. What Design Patterns are in action here?

b) Extend JUnit with Performance Measurement:

1. Add code to `ConditionalTestCase` to automatically *measure* execution time of test methods!
2. Where would you put the results?

c) Extend JUnit with Performance Testing:

1. Add code to `ConditionalTestCase` to automatically *test* execution time of test methods! If execution takes too long, throw an `AssertionFailedError` with a proper message!
2. Which Design Pattern are you using?

Exercise 3

a) Build a TestSuite tree by executing code like:

```
public static void main(String[] args) {
    TestSuite suite1 = new TestSuite(MyTestCase);
    TestSuite suite2 = new TestSuite(MyTestCase);
    TestSuite suite3 = new TestSuite(MyTestCase);

    // Don't uncomment next line ;- )
    // suite1.addTest(suite1);

    suite1.addTest(suite2);
    suite2.addTest(suite3);

    Test test = new MyTestCase("testMethod");
    suite1.addTest(test);

    test = new MyTestCase("testMethod");
    suite2.addTest(test);

    test = new MyTestCase();
    //this fails NullPointerException!
```

```
        //suite3.addTest(test);  
  
        junit.textui.TestRunner.run(suite1);  
    }
```

- b) 1. What happens if TestSuite adds itself?
2. What can one do to make code safer?
3. Is this a bug?
- c) 1. What happens if you execute code like `suite3.addTest(test);`?
2. Is this a bug?
3. What is the root cause for this behavior?

Exercise 4

- a) Implement a **TestExecutor** as an **Invoker** class of the Command Pattern.
- b) Implement the **TestExecutor** in such a way that it distributes TestCases of e.g. a TestSuite over multiple Threads (compare to Callable Pattern of previous exercise). Think of Thread-Safety (Hint TestResult is a joint resource and might be protected from concurrent access.)!

Hints

- Consult the literature!
- You can work in pairs, if you want!
- If you want to learn a Java API, look into the java docs!
- Always use the same familiar IDE (suggestion JetBrains IntelliJ or Eclipse)!